

Gemma 4 26B MoE on GKE G4 (RTX PRO 6000)

10K Input / 500 Output Concurrency Sweep (C=1 to 512) & Client Comparison (vLLM vs. SGLang)

Model Architecture

- Model: google/gemma-4-26B-A4B
- Type: Sparse MoE Architecture
- Total Params: 26 Billion
- Active Params: ~4.0B per token
- Depth: 30 Transformer Layers
- Attention: Tied K/V projections, Sliding Window + Global Attn
- Quantization: FP8 Dynamic

GKE G4 Infrastructure

- Cluster: pm-g4-sglang-cluster
- Zone: us-south1-a (GKE 1.35)
- Node: g4-standard-48 (1x GPU)
- GPU: NVIDIA RTX PRO 6000 Blackwell Server Edition
- GPU Memory: 96 GB GDDR7
- CPU/RAM: 48 vCPUs / 180 GB
- Parallelism: Tensor Parallel = 1

Serving & Clients

- Engine: vLLM (vllm-openai:gemma4)
- KV Cache: FP8 (--kv-cache-dtype)
- Context Limit: 12,288 Tokens
- Attention Backend: Triton Attn
- Workload: 10,240 In / 500 Out
- Sweep: Concurrency 1 to 512
- Clients Evaluated:
 - 1) vllm bench serve (Fixed)
 - 2) sglang.bench_serving (Uniform)

KEY TAKEAWAY: 1x RTX PRO 6000 GPU delivers 24,515 total tok/s (1,155 output tok/s) with <681ms TTFT up to C=64 on 10K prompts.

Single-GPU Blackwell Performance on 10K Context Workloads

Empirical results across Concurrency 1 to 512 on GKE g4-standard-48 (1x NVIDIA RTX PRO 6000 Blackwell, TP=1, FP8)

24,515 tok/s

Peak Total Token Throughput

- 24,515 tok/s with SGLang client (C=512)
- 22,312 tok/s with fixed 10K vLLM client
- Sustained on 1x RTX PRO 6000 GPU

178 – 681 ms

Sub-Second Median TTFT (C=1–64)

- 178ms median TTFT at C=1 (SGLang)
- 369ms median TTFT at C=1 (strict 10.2K)
- Stays under 681ms through C=64

1,155 tok/s

Peak Output Decode Throughput

- 1,155 output tok/s (SGLang C=512)
- 1,039 output tok/s (vLLM fixed 500 OSL)
- 119–128 tok/s single-user streaming

Sweet-Spot Regime: Concurrency 64 to 128

- Concurrency 64: Delivers 18,992 total tok/s (884 output tok/s) with 2.68s median TTFT under strict 10K/500 load, and 17,042 total tok/s (681ms median TTFT) under uniform load.
- Concurrency 128: Reaches 21,489 total tok/s (1,001 output tok/s) with 2.18s median TTFT.
- Balances maximum GPU tensor core saturation with responsive user experience.

Single-GPU Equivalence (g4-standard-48 vs. 384)

- Dedicated 1x GPU node (g4-standard-48) matches 8x GPU node (g4-standard-384, TP=1) within <0.5% across all concurrency points.
- Proves zero host CPU, NUMA, or PCIe bandwidth bottleneck on 1-GPU G4 slices.
- Enables fine-grained GKE horizontal pod autoscaling by single-GPU increments.

Why Gemma 4 26B MoE Excels on Single-GPU G4 & Workstations

Activating only ~4B parameters per token yields 2.5x–3x higher decode throughput than 31B Dense

Gemma 4 26B MoE vs. 31B Dense Architecture

- Sparse Mixture-of-Experts (MoE) Efficiency:
Gemma 4 26B has 26B total parameters but activates only ~4.0B–6.2B parameters per token (vs 31B active per token in Gemma 4 31B Dense).
- Overcoming Memory-Bandwidth Bottlenecks:
LLM token generation (decode) is strictly memory-bandwidth bound.
Reading 4B active weights per step yields 2.5x–3x higher decode tok/s.
- Compact Layer & KV Cache Footprint:
30 Transformer layers (vs 60 in 31B Dense) with tied K/V projections, sliding window + global attention reduces KV cache memory pressure.
- Massive Batch Headroom on 96GB GDDR7:
FP8 model weights (~26 GB) leave >60 GB VRAM for FP8 KV cache, enabling 512 concurrent 10K-context requests without preemption.

Cross-Platform Benchmark Comparison (26B MoE)

- Cloud GPU — 1x NVIDIA RTX PRO 6000 (96GB GDDR7, vLLM FP8):
 - Single-Stream (C=1): 109.5 – 118.0 decode tok/s (7.8ms TPOT)
 - Sweet Spot (C=128): 1,000.5 decode tok/s | 21,489 total tok/s
 - Peak Saturation (C=512): 1,154.8 decode tok/s | 24,515 total tok/s
- Local Workstation — Apple M4 Pro (48GB Unified, LiteRT-LM Q4_0):
 - Single-Stream Decode: ~60.2 tok/s (2x faster than 12B Dense @ 30.4)
 - 1K Prompt Prefill Throughput: ~659 tok/s | RAM: 2.3 – 25.6 GB
- Cloudtop CPU — AMD EPYC 7B13 (64-Core, llama.cpp AVX2 Q4_0):
 - Single-Stream Decode: 24.9 – 29.9 tok/s (vs 11.6 tok/s Dense)
 - Host RAM Footprint: 15.0 GB (@10 tok) to 42.5 GB (@64K context)

Gemma 4 26B vLLM Sweep: Strict Fixed 10,239 Input / 500 Output

Executed via vllm bench serve --random-range-ratio 0.0 --ignore-eos on 1x g4-standard-48 (TP=1, FP8)

Conc.	Prompts	Output Tok/s	Total Tok/s	Median TTFT	P99 TTFT	Median TPOT	Median E2E
1	8	109.45	2,350.85	368.5 ms	372.9 ms	8.41 ms	4.57 s
8	16	395.76	8,500.21	1,729.8 ms	2,674.3 ms	16.76 ms	10.09 s
16	32	555.73	11,935.96	2,096.3 ms	5,299.6 ms	25.00 ms	14.31 s
32	64	701.89	15,075.21	2,652.0 ms	10,464.2 ms	40.19 ms	22.62 s
64	128	884.24	18,991.66	2,675.6 ms	21,011.0 ms	68.08 ms	35.66 s
128	256	1,000.53	21,489.31	2,176.5 ms	42,330.3 ms	120.23 ms	62.69 s
256	512	1,020.99	21,928.85	37,146.3 ms	107,201.0 ms	144.94 ms	109.40 s
512	1024	1,038.83	22,311.93	166,089.8 ms	235,171.3 ms	145.36 ms	237.53 s

Strict Length Enforcement (--random-range-ratio 0.0): Every prompt processes strictly 10,739 tokens (10,239 in + 500 out). Highlighted rows (C=64, C=128) show optimal production sweet spot (>21.4K tok/s at 2.18s TTFT).

Gemma 4 26B vLLM Sweep: SGLang Client (Uniform Range)

Executed via `sglang.bench_serving --random-range-ratio 1 --ignore-eos` on identical vLLM G4 server

Conc.	Prompts	Output Tok/s	Input Tok/s	Total Tok/s	Median TTFT	Median TPOT	Median E2E
1	8	118.03	2,317.8	2,435.80	177.5 ms	7.80 ms	1.35 s
8	16	321.25	5,458.2	5,779.40	275.5 ms	19.05 ms	5.77 s
16	32	436.33	8,238.0	8,674.37	363.1 ms	29.11 ms	8.14 s
32	64	565.62	11,430.2	11,995.77	592.7 ms	45.60 ms	12.91 s
64	128	733.83	16,308.1	17,041.97	680.8 ms	68.08 ms	16.94 s
128	256	941.48	19,679.1	20,620.57	1,039.4 ms	98.68 ms	28.15 s
256	512	1,056.21	22,850.2	23,906.38	14,167.1 ms	146.21 ms	49.55 s
512	1024	1,154.84	23,360.5	24,515.32	67,969.7 ms	156.89 ms	95.17 s

Upstream Baseline Compatibility (--random-range-ratio 1): Samples prompt lengths uniformly from [0.5x, 1.0x] (~5,770 avg input / ~280 avg output tokens). Reaches 24,515.32 total tok/s and 1,154.84 output tok/s at C=512.

[🔗 Full SGLang Client Report: results/sqlang_benchmark_10k_500_report.md ↗](#)

Deep-Dive: vllm bench serve vs. sglang.bench_serving

Why prompt length sampling (--random-range-ratio) explains all throughput & latency differences

1. Prompt Sampling

- Different CLI Definitions:
The root cause of performance differences is how each client interprets --random-range-ratio.
- vllm bench serve ($r = 0.0$):
Samples $[L \cdot (1-r), L \cdot (1+r)]$.
Every request is strictly 10,239 input + 500 output = 10,739 tok.
- sglang.bench_serving ($r = 1$):
Clamps random sampling to $[0.5 * L, 1.0 * L]$.
Requests average ~5,770 input + ~280 output = ~6,050 tok/req (~44% fewer tokens per prompt).

2. Medium Load (C=8–128)

- vLLM Client Higher Tok/s:
From C=8 to C=128, vllm bench serve achieves higher Total Tok/s (8.5K–21.5K vs 5.8K–20.6K) and Output Tok/s (396–1,001 vs 321–941).
- Decode Amortization Effect:
In vllm bench serve, every prompt executes a full 500-token decode loop without early termination at 250–300 tokens.
- Continuous Batch Saturation:
Longer decode sequences amortize prefill cost and keep RTX PRO 6000 FP8 tensor cores saturated.

3. High Load (C=256–512)

- Prefill Queueing & TTFT:
Prefill time scales linearly with input length. At C=1, 10.2K tokens takes 368ms vs 178ms for 5.7K.
- Queue Wait Amplification:
At C=512 (max-num-batched-tokens=12288), queueing 10.2K prefills causes 2.4x longer queue waits (166s vs 68s median TTFT).
- E2E Latency Proportionality:
Decoding 500 tokens takes ~1.8x longer than decoding ~280 tokens at identical TPOT (~68ms @ C=64: 35.7s vs 16.9s E2E latency).

Production Sizing & Benchmark Methodology Guidelines

Recommended operating points on GKE g4-standard-48 and benchmark client best practices

Interactive (C=1–16)

- Target Workload: Real-time chat, coding assistants, interactive UI
- Median TTFT: 178 ms – 363 ms
- Generation: 35 – 128 tok/s/user
- Total Tok/s: 2.4K – 11.9K tok/s
- Delivers instant sub-400ms TTFT.

Sweet Spot (C=64–128)

- Target Workload: Multi-tenant API serving & agentic workflows
- Median TTFT: 0.68 s – 2.68 s
- Output Rate: 884 – 1,001 tok/s
- Total Tok/s: 17.0K – 21.5K tok/s
- Optimal HPA target per G4 GPU.

Batch Saturation (C=256–512)

- Target Workload: Offline document summarization & async pipelines
- Output Rate: 1,021 – 1,155 tok/s
- Total Tok/s: 21.9K – 24.5K tok/s
- Req Throughput: 4.4 – 4.5 req/s
- Maximizes tokens per GPU dollar.

When to Use vllm bench serve

- Flag: `--dataset-name random --random-input-len 10240 --random-output-len 500 --random-range-ratio 0.0 --ignore-eos`
- Best For: Strict SLA validation, worst-case KV cache sizing, and deterministic hardware stress testing where every request must hit exact context limits.

When to Use sglang.bench_serving

- Flag: `python3 -m sglang.bench_serving --dataset-name random --random-input-len 10240 --random-output-len 500 --random-range-ratio 1 --ignore-eos`
- Best For: Cross-engine comparisons against published SGLang benchmark reports and evaluating realistic variable-length traffic ([0.5x, 1.0x] distribution).